

```

const sharp = require("sharp");
const path = require("path");
const fs = require("fs");
const config = require("./config");

const W = config.MOCKUP_WIDTH;
const H = config.MOCKUP_HEIGHT;

function svgText(lines, opts = {}) {
  const { y = H / 2, fontSize = 60, color = "#1a1a1a", weight = "normal", spacing } = opts;
  return lines.map((line, i) => `
    <text x="50%" y="${y + i * spacing}" font-family="Georgia, serif"
      font-size="${fontSize}" font-weight="${weight}" fill="${color}"
      text-anchor="middle" dominant-baseline="middle">${line}</text>`
  ).join("");
}

function makeSvgOverlay(content, bg = "transparent") {
  return Buffer.from(`
    <svg width="${W}" height="${H}" xmlns="http://www.w3.org/2000/svg">
      <rect width="${W}" height="${H}" fill="${bg}" />
      ${content}
    </svg>
  `);
}

async function mockupClean(designPath, shopName, outputDir) {
  const padding = 300;
  const designSize = W - padding * 2;
  const overlay = makeSvgOverlay(`
    ${svgText([shopName.toUpperCase()], { y: 120, fontSize: 52, color: "#888888",
    ${svgText(["INSTANT DIGITAL DOWNLOAD"], { y: H - 110, fontSize: 44, color: "#
    <rect x="${padding - 20}" y="${padding - 20}" width="${designSize + 40}" heig
      fill="none" stroke="#eeeeee" stroke-width="3"/>
  `);
  const design = await sharp(designPath)
    .resize(designSize, designSize, { fit: "contain", background: { r: 255, g: 25
    .toBuffer();
  const out = path.join(outputDir, "mockup_1_clean.jpg");
  await sharp({ create: { width: W, height: H, channels: 3, background: { r: 255,
    .composite([
      { input: design, top: padding, left: padding },
      { input: overlay,
    ].jpeg({ quality: 95 })
    .toFile(out);
  return out;
}

```

```
}
```

```
async function mockupDark(designPath, title, outputDir) {
  const padding = 350;
  const designSize = W - padding * 2;
  const overlay = makeSvgOverlay(`
    ${svgText([title], { y: 140, fontSize: 58, color: "#ffffff", weight: "bold",
    ${svgText(["♦ High Resolution ♦ Commercial License ♦ Instant Download"],
  `);
  const design = await sharp(designPath)
    .resize(designSize, designSize, { fit: "contain", background: { r: 30, g: 30,
    .toBuffer();
  const bg = makeSvgOverlay("", "#1e1e1e");
  const out = path.join(outputDir, "mockup_2_dark.jpg");
  await sharp(bg).resize(W, H)
    .composite([ { input: design, top: padding, left: padding }, { input: overlay,
    .jpeg({ quality: 95 })
    .toFile(out);
  return out;
}
```

```
async function mockupFeatures(designPath, outputDir) {
  const designSize = 1100;
  const designLeft = (W - designSize) / 2;
  const designTop = 280;
  const features = [
    "✓ 2000 x 2000 px | 300 DPI",
    "✓ PNG with transparent background",
    "✓ Commercial use license included",
    "✓ Instant download after purchase",
  ];
  const featureSvg = features.map((f, i) => `
    <text x="50%" y="${designTop + designSize + 120 + i * 90}"
      font-family="Georgia, serif" font-size="46" fill="#333333"
      text-anchor="middle" dominant-baseline="middle">${f}</text>`
  ).join("");
  const overlay = makeSvgOverlay(`
    ${svgText(["WHAT YOU GET"], { y: 150, fontSize: 56, color: "#111111", weight:
    <rect x="${designLeft - 10}" y="${designTop - 10}" width="${designSize + 20}"
      rx="20" fill="#f5f5f5" stroke="#e0e0e0" stroke-width="2"/>
    ${featureSvg}
  `);
  const design = await sharp(designPath)
    .resize(designSize, designSize, { fit: "contain", background: { r: 245, g: 24
    .toBuffer();
  const out = path.join(outputDir, "mockup_3_features.jpg");
  await sharp({ create: { width: W, height: H, channels: 3, background: { r: 255,
```

```

        .composite([ { input: overlay, top: 0, left: 0 }, { input: design, top: design
        .jpeg({ quality: 95 })
        .toFile(out);
    return out;
}

```

```

async function mockupColorPop(designPath, outputDir) {
    const padding = 320;
    const designSize = W - padding * 2;
    const overlay = makeSvgOverlay(`
        <defs>
            <linearGradient id="bg" x1="0%" y1="0%" x2="100%" y2="100%">
                <stop offset="0%" style="stop-color:#fde8d8;stop-opacity:1"/>
                <stop offset="100%" style="stop-color:#fbd5e8;stop-opacity:1"/>
            </linearGradient>
        </defs>
        <rect width="${W}" height="${H}" fill="url(#bg)"/>
        ${svgText(["DIGITAL CLIPART"], { y: 130, fontSize: 64, color: "#c0608a", weight: "bold" })}
        ${svgText(["Perfect for cards, crafts & invitations"], { y: H - 120, fontSize: 14 })}
    `);
    const design = await sharp(designPath)
        .resize(designSize, designSize, { fit: "contain", background: { r: 253, g: 231, b: 253 } })
        .toBuffer();
    const out = path.join(outputDir, "mockup_4_colorpop.jpg");
    await sharp(overlay).resize(W, H)
        .composite([ { input: design, top: padding, left: padding } ])
        .jpeg({ quality: 95 })
        .toFile(out);
    return out;
}

```

```

async function generateAllMockups(designPath, { title, shopName }, outputDir) {
    if (!fs.existsSync(outputDir)) fs.mkdirSync(outputDir, { recursive: true });
    const mockups = await Promise.all([
        mockupClean(designPath, shopName, outputDir),
        mockupDark(designPath, title, outputDir),
        mockupFeatures(designPath, outputDir),
        mockupColorPop(designPath, outputDir),
    ]);
    return mockups;
}

```

```

module.exports = { generateAllMockups };

```